

全栈学习深化指南

配套《个人定制学习路线》· 非 AI 部分细节展开

覆盖范围：前端工程化 / 后端服务化 / 架构师方向 / 鸿蒙 / 工具与方法论
与 screen-ai-deepdive.pdf 互为姊妹篇，共同构成完整学习手册

一	学习底层逻辑：学得快 vs 学得慢的 4 个差异
二	阶段一深化：现代前端工程化（Vue 3 + TS + Vite + Tailwind）
三	阶段二深化：后端服务化（FastAPI + SQLAlchemy + Redis + Docker）
四	深化路线 B：架构师方向（系统设计 + 分布式 + 云原生）
五	深化路线 C：鸿蒙生态（ArkTS + ArkUI 转型经验）
六	Git 协作工作流（个人开发与团队协作）
七	工具链使用经验（Cursor / Apifox / 终端 / DB 工具）
八	学习方法论（节奏 / 输出 / 防 burnout）
九	跨方向踩坑经验集（15 个常见问题）
十	精选资源清单（按方向分类）

一、学习底层逻辑：学得快 vs 学得慢的 4 个差异

同样学 Vue / FastAPI / Docker，有人 2 周搞定，有人 2 个月还卡着。技术差异极小，方法差异极大。

4 大差异

1. 是否带"用途感"学	学得慢：按章节顺序看文档，逐字啃完。 学得快：先想清楚"我要做一个 X"，按需查文档，做完再回头补漏。 差距倍率：3-5 倍。带任务的学习记忆留存率高 80%。
2. 是否会"敲一遍"	学得慢：看视频 + 收藏文档，认为"看懂了"。 学得快：每个概念至少手敲 3 遍，故意改坏看报什么错。 本质：写过的代码和看过的代码，在大脑中是完全不同的两套表征。
3. 是否敢于"读源码 / 读报错"	学得慢：报错就复制去搜，搜不到就放弃。 学得快：报错从最后一行往上读，看 stack trace 自己找原因；遇到 API 不懂直接读源码。 关键认知：报错信息是最好的老师，源码是最权威的文档。
4. 是否有"输出回路"	学得慢：自己学自己懂，没人验证。 学得快：每周写一篇笔记 / GitHub 提交 / 给别人讲。 科学依据：费曼学习法 — 能给小白讲明白 = 真的懂了。

心态层面：3 个反直觉真相

1. 不要追求"全懂再用"	Vue 文档 200 页，你看完 30% 就能开始做项目。剩下 70% 在用的过程中自然就懂了。"等学完再做"是最大的陷阱——永远学不完。
2. 卡 1 小时就该求助	独立思考 1 小时还卡着 → 直接问 AI / 同行 / 社区。 不是认输，是节省时间。你不是在锻炼"硬抗 bug"的能力，是在学技术。卡的时间长成本极高。
3. 接受"暂时性垃圾代码"	前 3 个项目的代码注定丑陋。逼自己写"完美代码"反而完成不了项目。 原则：先跑通，再重构，再优化。从 0 到 1 永远比 1 到 100 重要。

一句话：学习的本质不是"输入知识"，是"建立和环境的反馈回路"。能让你拿到反馈的所有动作（写代码、报错、被审、被用）都比读书强。

你已有 HTML/CSS/JS 基础，本章解决"为什么要工程化、Vue 3 / TS / Vite / Tailwind 各自的角色"。

从"切图仔"到"前端工程师"的本质跨越

原始 jQuery 时代	直接操作 DOM、全局变量、IDE 智能提示弱。300 行以上就难以维护。
现代工程化	组件化（复用 UI）+ 响应式（数据驱动 DOM）+ 模块化（一个文件一个职责）+ 类型化（编辑器和 CI 帮你抓 bug）+ 工具化（自动构建、热更新、Tree-shaking）。
为什么必学	2026 年所有前端岗都默认要求 Vue/React + TS + Vite。即使你做 WordPress / 切图，懂工程化 = 工资上限直接翻倍。

Vue 3 学习路径（推荐顺序，避免乱学）

顺序	内容	关键点
1	响应式：ref vs reactive	ref 用于基本类型，reactive 用于对象。混用是新手最大错误源。
2	模板语法：v-if / v-for / v-model	v-for 必须加 :key（用 id 不要用 index）。v-if 与 v-for 不要同级。
3	组合式 API：setup + computed + watch	computed 是缓存的，watch 是副作用。能用 computed 就别用 watch。
4	组件通信：props / emit / provide-inject	父子用 props/emit，跨层用 provide/inject，全局状态用 Pinia。
5	生命周期钩子	onMounted（DOM 已挂载）和 onBeforeUnmount（清理）最常用。
6	路由 Vue Router 4	懒加载组件（() => import）、路由守卫、动态参数。
7	状态管理 Pinia	比 Vuex 简单 10 倍。defineStore + state/getters/actions 三件套。
8	异步组件 + Suspense	大组件懒加载。配合骨架屏可显著提升首屏体验。

TypeScript 8 大核心（先掌握这些，剩下用到再学）

1. 基本类型注解	let name: string = "Tom" — 但能省则省，TS 会自动推断。
2. interface vs type	interface 描述对象形状（推荐默认）；type 描述任意类型（联合/交叉/工具）。
3. 联合类型	type Status = "loading" "ok" "error" — 替代 enum，更轻量。
4. 可选与默认值	name?: string（可选）；name: string = "默认"。
5. 泛型	函数/接口的参数类型化。Array、Promise 是最常见用法。
6. as 类型断言	当你比 TS 更确定类型时用。避免滥用 as any（杀手锏，但杀的是自己）。

7. 内置工具类型

Partial / Required / Pick / Omit / Record — 处理对象类型的瑞士军刀。

8. 严格模式 strict

tsconfig 里开 strict: true。一开始报错很多，但能避免 90% 的运行时 bug。

Vite vs Webpack：你需要知道的真相

为什么 Vite 快	开发模式：原生 ESM + 按需编译（你访问哪个文件才编译哪个）；生产模式：用 Rollup 打包。冷启动比 webpack 快 10-100 倍。
什么时候不用 Vite	老项目 webpack 改造成本高于价值；插件生态 Vite 略小但 2026 年已经够用。新项目无脑选 Vite。
必须会的 5 件事	1) 创建项目 <code>npm create vue@latest</code> 2) 配置代理（ <code>vite.config.ts</code> 里的 <code>server.proxy</code> 解决跨域） 3) 环境变量（ <code>.env.development</code> / <code>.env.production</code> ） 4) 别名（ <code>@</code> → <code>src</code> ） 5) 生产打包 <code>npm run build</code> → <code>dist</code> 目录

Tailwind CSS 实战经验

范式转变	从"写 CSS 类名 + 文件"→"直接在 HTML 写工具类"。 初学者觉得丑，用熟了就回不去——可视化反馈太快了。
必背 30 个类	<code>flex</code> / <code>items-center</code> / <code>justify-between</code> / <code>gap-4</code> / <code>p-4</code> / <code>m-4</code> / <code>w-full</code> / <code>h-screen</code> / <code>text-sm</code> / <code>text-gray-600</code> / <code>bg-white</code> / <code>rounded-lg</code> / <code>shadow</code> / <code>border</code> / <code>hover:</code> / <code>md:</code> / <code>lg:</code>
响应式断点	移动优先：默认样式给手机， <code>md:/lg:</code> 加桌面端覆盖。 例： <code>class="w-full md:w-1/2"</code> → 手机占满，桌面占一半。
避坑	1) 不要混用 Tailwind + 大量自定义 CSS，会乱 2) 重复的组合用 <code>@apply</code> 抽取或直接做组件 3) Tailwind 配置文件 <code>tailwind.config.js</code> 设计 token 一次，全项目复用
扩展库	<code>shadcn/ui</code> (React) / <code>shadcn-vue</code> (Vue) — 复制粘贴式组件库，2026 年生态最火，UI 直接能用。

前端工程化必备 5 件套

ESLint	代码规范检查。配 <code>vue</code> 推荐规则 + <code>ts</code> 推荐规则。保存自动 <code>fix</code> 。
Prettier	代码格式化。统一缩进、引号、分号。团队协作必备。
Husky + lint-staged	<code>git commit</code> 前自动跑 ESLint+Prettier。脏代码进不了仓库。
Commitizen	规范 <code>commit message</code> （ <code>feat/fix/docs/refactor...</code> ）。便于自动生成 changelog。
VitePWA / Compression	生产构建插件。 <code>gzip</code> 压缩、PWA 离线支持，部署前打开。

第一个完整前端项目：练手方案

"待办事项 + 番茄钟"应用（自己用得到，又覆盖所有基础点）

技术栈	Vue 3 + TS + Vite + Pinia + Vue Router + Tailwind + Axios
功能点	增删改查待办、分类标签、本地存储 (localStorage)、番茄钟倒计时、统计图表、暗黑模式、PWA 离线
覆盖能力	组件拆分 + Pinia 状态 + 路由跳转 + 数据持久化 + 副作用管理 + 响应式样式 + 工程化
部署	Vercel / Netlify 一键部署（连 GitHub 仓库自动 CI/CD）
耗时	认真做 1-2 周。做完你的简历就有了第一个"能展示"的项目。

阶段一结束自检

能用 Vue 3 Composition API 写出可复用的组件
会用 TypeScript 给组件 props / 函数参数加类型，不写 any
能用 Pinia 管理跨组件状态，会处理异步 action
配过 Vite 代理解决跨域，会配置环境变量
用 Tailwind 做出在手机和桌面都好看的页面（移动优先）
至少完整部署过 1 个项目（有公网 URL）

你已有 PHP 经验，本章解决"为什么转 Python + FastAPI、它和 PHP 的本质差异"。

PHP 转 Python/FastAPI：你的优势与转换点

维度	PHP（你已有）	Python + FastAPI（要学）
运行模型	每次请求新进程（短生命周期）	常驻进程（长生命周期）。意味着你可以缓存、定时任务、长连接
类型系统	弱类型 + 运行时检查	静态类型 (TypeHints) + Pydantic 强校验。Bug 少 70%
异步模型	同步阻塞（请求一个等一个）	async/await 原生异步。一个进程能处理上千并发
ORM	Laravel Eloquent / Doctrine	SQLAlchemy（更强大但学习曲线略陡）
模板引擎	Blade / Twig	前后端分离，后端只输出 JSON，不再写模板
部署	Apache/Nginx + PHP-FPM	Docker + Uvicorn/Gunicorn + Nginx 反代

FastAPI 8 大核心特性

1. 路径操作装饰器	@app.get("/users/{id}") — 一行声明 RESTful 接口。比 Flask 简洁 50%。
2. Pydantic 数据校验	class UserCreate(BaseModel) — 自动从 JSON 校验+反序列化。客户端瞎传立刻报 422。
3. 依赖注入 Depends	把"获取当前用户/数据库连接"封装成依赖，多接口复用。FastAPI 灵魂特性。
4. 自动文档 OpenAPI	/docs 路由自动生成 Swagger UI；/redoc 生成更美观的文档。零成本 API 文档。
5. 异步路由 async def	路由函数加 async，可以并发 await 数据库/外部 API。性能提升 3-10 倍。
6. 中间件 Middleware	统一处理 CORS、日志、限流、错误。@app.middleware("http")。
7. 后台任务 BackgroundTasks	接口立即响应，耗时操作（发邮件/写日志）后台执行。简单场景代替 Celery。
8. WebSocket 原生支持	@app.websocket("/ws") — 流式 AI 输出、聊天室必备。Flask 没有。

SQLAlchemy 实战要点

Model 定义	class User(Base): id = Column(Integer, primary_key=True). 用 declarative_base。
Session 管理	每个请求开一个 session，用完关闭。FastAPI 用 Depends 包装最优雅。
查询模式	session.query(User).filter(User.email == x).first() — 链式调用。2.0 版改用 select() 函数式。
关联关系	relationship() + ForeignKey。lazy="select" 懒加载 vs lazy="joined" 即时加载。

N+1 陷阱

循环查关联数据 = 性能黑洞。用 joinedload 或 selectinload 提前 join。

Alembic 迁移

版本化数据库 schema。alembic revision --autogenerate 自动生成迁移脚本。

JWT 认证完整流程

为什么不用 Session	Session 需要服务端存储（依赖 Redis/DB），多机部署要共享。JWT 是无状态 Token，分布式天然友好。
完整流程	1) 登录验证账密 → 服务端生成 JWT（含 user_id + 过期时间，用 secret 签名） 2) 返回 token 给前端，前端存 localStorage / cookie 3) 后续请求加 Header: Authorization: Bearer {token} 4) FastAPI 依赖注入解析 token → 拿 user_id → 查用户
安全要点	1) SECRET_KEY 放 .env，不能进 git 2) HTTPS 必备，HTTP 下 token 会被中间人窃听 3) Token 短期有效（15-60 分钟），用 refresh_token 续期 4) 密码必须 bcrypt 哈希存储，永远不存明文
Python 工具	python-jose（JWT 编解码）+ passlib[bcrypt]（密码哈希）。FastAPI 官方教程有完整示例。

Redis 6 大典型场景

场景	数据结构	示例
缓存	String	高频接口结果缓存 5 分钟。SET key value EX 300
分布式锁	String + SETNX	防止任务被多次执行（如订单去重）
计数器	INCR	点赞数、访问数。原子递增不需要事务
排行榜	ZSET	游戏积分榜、热门商品 TopN。ZADD + ZREVRANGE
消息队列	List / Stream	LPUSH + BRPOP 简单队列；Stream 更可靠
Session 存储	Hash	多机共享登录状态

Docker / Compose 工程化

Dockerfile 基础	FROM python:3.11-slim → COPY → pip install → CMD uvicorn ... → 镜像大小控制在 200MB 内。
多阶段构建	编译阶段 + 运行阶段分离。前端项目从 node → nginx 两阶段。镜像可缩小 5-10 倍。
docker-compose.yml	一个 yml 起多服务（前端 + 后端 + MySQL + Redis）。开发环境一行 docker compose up 启动全栈。
数据卷 volumes	数据库数据必须挂卷，否则容器删了数据没了。这是新手最常踩的灾难。
网络 networks	同 compose 内服务直接用服务名互相访问（如 mysql:3306）。不需要 IP。

生产环境注意

- 1) 不要在容器里直接改代码
- 2) 日志输出到 stdout 让 docker logs 收集
- 3) 用 .env 文件管理配置，不要写死在 yml

MySQL 进阶（你已有基础）

索引底层	B+ 树。主键索引 = 数据本身（聚簇索引）。辅助索引存的是主键值，所以要回表。
索引怎么建	WHERE/ORDER BY/JOIN 用到的列加索引。最左前缀原则：联合索引 (a,b,c) 能用上 a / a,b / a,b,c。
EXPLAIN	看执行计划。type=ref 是好的，type=ALL 是全表扫描灾难。rows 估算扫描行数。
事务隔离级别	MySQL 默认 RR (可重复读)。你需要知道脏读/不可重复读/幻读是什么，面试常考。
连接池	生产必备。SQLAlchemy 的 pool_size + max_overflow + pool_recycle 三参数要会调。
分页优化	OFFSET 大了变慢。改用游标分页 (WHERE id > last_id ORDER BY id LIMIT 20) 。

阶段二结束自检

能不查文档写出 FastAPI + SQLAlchemy 的 CRUD 接口

理解 async/await 在什么场景能提升性能，什么场景没用

会用 Pydantic 做数据校验，能定义嵌套模型

完整实现过 JWT 登录注册 + 密码哈希

在 Redis 至少使用过 3 种数据结构解决实际问题

能写 Dockerfile + docker-compose.yml 一键起多服务

会用 EXPLAIN 分析慢查询并加索引优化

适合人群：享受从 0 到 1

做产品的全过程、想成为技术负责人。这个方向的核心是“系统设计能力”——从单机到百万用户的演进路径。

系统设计 5 大支柱

1. 可扩展性 Scalability	系统能否线性增加机器扛更多流量？关键：无状态服务 + 水平拆分 + 异步解耦。
2. 可用性 Availability	99.9% / 99.99% SLA 意味着每年宕机多少分钟？关键：冗余 + 故障转移 + 健康检查。
3. 一致性 Consistency	强一致 vs 最终一致。CAP 定理：网络分区时必须选在 C 和 A 中选一个。
4. 性能 Performance	P50/P99/P999 延迟。重点不是平均，而是长尾。一个慢接口能拖垮整个系统。
5. 可维护性 Maintainability	日志、监控、链路追踪、文档。系统能否在凌晨 3 点被人快速排查？

分布式核心 8 概念

负载均衡	Nginx / LVS / 云厂商 SLB。轮询 / 加权 / 一致性哈希等策略。
服务发现	服务实例动态变化时，调用方如何找到。Consul / Etcd / Nacos。
消息队列	异步解耦、流量削峰、最终一致。Kafka（吞吐量）/ RabbitMQ（可靠性）/ Redis Stream（轻量）。
分布式锁	多机抢资源。Redis SETNX（简单）/ Redlock（更可靠）/ Zookeeper（最严格）。
分布式事务	Saga 模式：长流程拆步骤，每步可补偿。比 2PC 实用得多。
一致性哈希	数据分片时减少节点变动带来的迁移。Redis Cluster / 一致性哈希环。
限流熔断	Sentinel / Hystrix。令牌桶 / 漏桶算法。雪崩防护必备。
链路追踪	Jaeger / SkyWalking。一个请求跨多个服务的完整路径可视化。

数据库进阶决策树

场景	推荐方案	决策点
单机 MySQL 撑不住读	主从 + 读写分离	读多写少时优先。一主多从。
单表千万行慢	分库分表 + ShardingSphere	冷热数据分离比直接分表更简单。
多数据中心	TiDB / OceanBase (NewSQL)	原生分布式，兼容 MySQL 协议。
超大规模时序	InfluxDB / TimescaleDB	监控、IoT 数据。普通数据库扛不住。
复杂搜索	Elasticsearch + 同步管道	电商搜索、日志聚合标配。
图数据	Neo4j / NebulaGraph	社交关系、知识图谱场景。

云原生学习顺序

阶段 1 : Docker 熟练	Dockerfile / 镜像优化 / docker-compose。这是前置。
阶段 2 : K8s 基础	Pod / Deployment / Service / Ingress 四件套。用 minikube 本地玩。
阶段 3 : Helm Chart	把应用打包成可复用的 chart。生产部署标配。
阶段 4 : GitHub Actions	CI/CD 自动化。代码 push 自动构建镜像 + 部署。
阶段 5 : Terraform IaC	基础设施代码化。云资源（VPC、ECS、RDS）用代码声明。
阶段 6 : 可观测性	Prometheus + Grafana + ELK + Jaeger。监控全家桶。

架构师方向核心建议：不要孤立学这些技术。最有效的方法是参与一个真实有规模的开源项目，看大佬们如何在真实场景中应用。推荐：Apache Dubbo、TiDB、Kratos 等中国主导的开源项目。

为什么 2026 年值得入场鸿蒙

市场现状	2025 年华为 HarmonyOS NEXT 全面去 Android 化。设备规模破 10 亿，应用生态还在快速建设中。
稀缺程度	原生鸿蒙开发者目前 < 50 万，国内招聘需求每年 50 万+。供给严重不足。
薪资倒挂	同等经验，鸿蒙开发者比 Android 高 30-50%（截至 2026 年初）。
适合谁	有 TS / Vue / React 基础的人。ArkTS 学习成本 1-2 周。
不适合谁	没有前端基础、不想接触移动端、对华为生态有偏见的人。

ArkTS vs TypeScript：差异点

增量	装饰器（@State / @Link / @Prop）+ 声明式 UI 语法。
删减	禁止 any、禁止动态属性、禁止运行时类型修改。比 TS 更严格。
设计哲学	TypeScript 是给 JS 加类型；ArkTS 是为了让编译器能做更激进的优化（AOT 编译）。
迁移成本	熟练 TS 的人 1-2 周完全适应。语法上几乎是 TS 的子集 + 鸿蒙特化。

ArkUI 核心思想（类比理解）

声明式 UI	类似 SwiftUI / Flutter / Jetpack Compose。UI = f(state)。状态改变 → UI 自动重渲染。
Column / Row	类比 Flutter Column/Row 或 CSS Flex。布局核心。
Stack	类比绝对定位。多层叠加。
@State	组件内部状态。改了自动触发 UI 更新。类比 Vue ref。
@Link	父子双向绑定。类比 Vue v-model。
@Prop	父传子单向。类比 Vue props。
@Provide / @Consume	跨层级传值。类比 Vue provide/inject。

12 周转型路线

周次	内容	产出
1-2	ArkTS 语法 + DevEco Studio 环境搭建	跑通 Hello World + Codelab
3-4	ArkUI 核心组件 + 状态管理	单页应用（如天气卡片）
5-6	路由 + 网络请求 + 数据存储	完整应用：多页 + API 调用

7-8	多设备适配（手机/平板/折叠屏）+ 原子化服务	能适配手机平板，发布到模拟器
9-10	原生能力调用（相机/位置/通知）	一个有原生特性的应用
11-12	AppGallery Connect 发布	上架一个真实鸿蒙应用

鸿蒙学习资源

官方文档	developer.huawei.com/consumer/cn — 中文权威，必读。
Codelab	developer.huawei.com/consumer/cn/codelabsPortal — 实战教程，跟着抄一遍。
B 站	"HarmonyOS 开发者"官方账号 + "硬核老王"等技术博主。
社区	devhub.huawei.com / 51CTO 鸿蒙专区。问答活跃度还在爬升。
GitHub	OpenHarmony 官方仓库。想理解底层的人看。

六、Git 协作 workflow

80% 的人 Git 只会 add/commit/push。剩下 20% 的命令是工程师与码农的分水岭。

个人开发必会 12 命令

git status / git log --oneline	看当前状态 / 紧凑日志。每次操作前 status 一下。
git diff / git diff --staged	看未暂存改动 / 已暂存改动。commit 前必看。
git add -p	交互式分块提交。一个文件里多个修改拆成多个 commit。神技。
git commit --amend	修改最后一次 commit (信息或内容)。已 push 的不要乱用。
git stash / git stash pop	临时收起未完成改动，去做别的，做完再 pop 回来。
git reset HEAD~1	撤销最近一次 commit (保留改动)。--hard 慎用 (彻底丢弃)。
git rebase -i HEAD~5	交互式合并/调整最近 5 个 commit。整理历史必备。
git cherry-pick	把别的分支的某个 commit 摘过来。修 bug 跨分支同步用。
git reflog	后悔药。所有操作记录，误删 commit 也能找回。
git tag v1.0.0	打版本标签。发版时用。
git remote -v / git fetch	看远程仓库 / 拉远程更新但不合并。
git branch -d / -D	删分支。小 d 安全，大 D 强删 (未合并也删)。

分支策略对比

策略	适合	核心思想
Trunk-Based	小团队 / 持续部署	所有人短期分支 + 频繁合主分支。GitHub/Google 等用。
Git Flow	中大型 / 版本化产品	main / develop / feature / release / hotfix 五分支。复杂但严谨。
GitHub Flow	SaaS 产品 / 团队 10 内	只有 main 和 feature。最简洁，事实标准。
GitLab Flow	需要环境分支	main + production / staging 环境分支。

Commit Message 规范 (Conventional Commits)

格式	<type>(<scope>): <subject>
type	feat (新功能) / fix (修 bug) / docs (文档) / refactor (重构) / test (测试) / chore (杂项)
示例	feat(auth): 添加 JWT 刷新机制 fix(api): 修复用户列表分页 OFFSET 越界

好处	1) 自动生成 CHANGELOG 2) 一眼看出改动性质 3) 触发 CI 不同流程 (feat → minor 版本号)
工具	commitlint + husky 强制规范 ; commitizen 交互式填写。

Pull Request / Code Review 礼仪

PR 不要太大	理想 < 400 行。1000+ 行的 PR 没人能认真审。拆成多个小 PR。
PR 描述要全	Why (为什么改) + What (改了什么) + How (怎么测的) + 截图 (如果是 UI) 。
审别人代码	关注架构、命名、边界、安全。不要纠结格式 (ESLint 干的活) 。
收到 review	感谢，不抬杠。不同意有理由就解释。承认错误是工程师的基本素养。

七、工具链使用经验

好工具能 10 倍提效。但工具是放大器：用好放大能力，用错放大焦虑。

编辑器选型 2026

Cursor (推荐)	VS Code 分叉 + 原生 AI。Tab 补全、Cmd+K 改代码、Cmd+L 对话。2026 年新人首选。
VS Code + Copilot	GitHub 官方 AI 插件。生态最大。如果公司用 GitHub Enterprise，无脑选这个。
JetBrains 全家桶	WebStorm/PyCharm。重型 IDE，重构和静态分析最强。已经熟悉的人不必换。
Zed	极简 + 协作友好 + Rust 写的快。前沿尝鲜可以试。
Vim/Neovim	极客必学（哪怕只是基础）。SSH 连服务器时唯一选项。

AI 辅助编程使用经验

好用场景	1) 模板代码（CRUD、表单校验） 2) 翻译代码（Python → Go） 3) 单元测试生成 4) 解读陌生代码 5) 错误诊断（贴报错给 AI）
不好用场景	1) 业务逻辑（AI 不懂你的业务） 2) 架构决策（AI 没全局上下文） 3) 优化已有代码的可读性（AI 风格僵化） 4) 安全敏感代码（AI 偶尔会写出漏洞）
使用心法	把 AI 当作“全栈但平庸的同事”。简单任务交给它批量处理，复杂任务自己上+让它审。永远要 review 它的输出，不要直接 Tab 接受看都不看。
反模式	"AI 依赖症"：所有问题问 AI 不查文档，几个月后基础概念全模糊。 建议：AI 帮你写，但你必须能解释每一行。

其他必备工具

类别	推荐工具	用途与经验
API 调试	Apifox / Postman	Apifox 国产更好用。支持 Mock、自动化测试、文档同步。
数据库管理	TablePlus / DBeaver / DataGrip	TablePlus 颜值高（Mac 收费）；DBeaver 免费功能全。
终端	iTerm2 (Mac) / Windows Terminal + WSL2	配合 oh-my-zsh + zsh-autosuggestions 体验起飞。
HTTP 客户端	curl / httpie	curl 是底层标准，httpie 语法更人性化。CI/CD 必备。
JSON 处理	jq	命令行处理 JSON。echo "..." jq ".data[].name"。
SSH 跳板	autossh / ssh-config	~/.ssh/config 配 alias，ssh prod 直连。

进程管理	pm2 / supervisor / systemd	pm2 适合 Node ; systemd 是 Linux 标配 ; Docker 优先。
性能分析	htop / btop / iostat	htop 看进程 ; iostat 看磁盘 IO ; netstat / ss 看连接。

终端配置：让你看起来像高手

Zsh + Oh My Zsh	Mac 默认。主题推荐 powerlevel10k (极简版 robbyrussell) 。
插件三件套	1) zsh-autosuggestions (自动建议历史命令) 2) zsh-syntax-highlighting (语法高亮) 3) fzf (模糊搜索文件/历史, Ctrl+R 体验飞升)
alias	常用命令缩写。alias gs="git status" ; alias dc="docker compose" 。
字体	JetBrains Mono / Fira Code (支持连字 ligature) 。

八、学习方法论

在职学习节奏控制（每天 2-3 小时）

时段	建议内容	原则
周一晚（1.5h）	看视频 / 读文档 / 新概念输入	低强度日，只看不写
周二晚（2h）	当天敲代码实践（昨天看的）	关键：动手验证
周三晚（1.5h）	继续昨天 + 复盘卡点	昨天卡哪儿了今天解决
周四晚（2h）	推进项目功能或专项练习	聚焦一件事
周五晚（1h）	整理本周笔记 / 写学习总结	哪怕 100 字也要写
周六上午（3h）	大块时间：项目 / 难题 / 读源码	需要深度专注的事放这里
周日	休息或自由探索，不强制	必须留一天充电

防 burnout（学习倦怠）5 原则

1. 不要日更	每天硬学 4 小时不如隔天学 6 小时。大脑需要消化时间。
2. 留 buffer 周	每 4 周留 1 周不学新东西，只巩固/做项目。规划外的事永远比想象的多。
3. 卡 2 小时必须停	卡 2 小时没进展 → 立刻问 AI / 同行 / 社区 / 转任务。硬扛是浪费生命。
4. 设置里程碑庆祝	完成一个项目 → 吃顿好的 / 买个礼物。正反馈机制对长期学习极其重要。
5. 找伙伴	加 1-2 个学习群（不要超过 5 个）。有压力，但别变成信息焦虑源。

"看似学了但其实没学"的 5 个信号

信号 1	"我看懂了这个视频" — 但合上视频写不出代码。纠正：暂停视频自己敲一遍。
信号 2	收藏夹一堆文章从没看过。纠正：清空收藏夹，只留近 1 周要看的。
信号 3	学了 X 个月还没有可展示的作品。纠正：暂停学习，2 周内必须做出一个 demo。
信号 4	能复述概念但解释不了"为什么"。纠正：用费曼法 — 假装给小白讲。
信号 5	换个相似场景就不会做了。纠正：刻意变形练习，故意改条件。

输出习惯：选一个就够

GitHub 提交	每天至少 1 个 commit（绿色方块刷成林）。最低成本输出。
技术博客	掘金 / 个人站。每周 1 篇 500 字以上。
社区回答	V2EX / Stack Overflow / 知乎技术回答。被采纳是高质量反馈。
B 站视频	录屏讲技术。门槛高但效果最深刻。

给他人讲

现实生活中找听众（同事、朋友、伴侣）。能让外行听懂 = 你真懂。

方法论一句话总结：你不是在"准备某天能开始工作"，你从今天起就是工程师，只是技能在每天叠加。带这种心态学习，效率与心理状态都会完全不同。

九、跨方向踩坑经验集（15 个真实问题）

1. 前端：Vue 响应式失效	直接给 reactive 对象赋值整对象（state = newObj）→ 失去响应式。 解：Object.assign(state, newObj) 或者用 ref + .value。
2. 前端：v-for + v-if 同级	v-for 优先级高于 v-if，导致每次都遍历全部再过滤。 解：用 computed 先过滤数组，或包一层 template。
3. 前端：跨域 CORS	开发环境 OK 生产坏。本质是 Vite 代理只在开发生效。 解：生产必须后端开 CORS 或用 Nginx 反代统一域名。
4. 前端：移动端 100vh 抖动	iOS Safari 滚动时地址栏出现 → 100vh 跳变。 解：用 100dvh（dynamic viewport height）或 JS 监听 resize 设置 CSS 变量。
5. 后端：FastAPI async 卡死	async 函数里调用同步阻塞库（如 requests）→ 整个事件循环卡住。 解：用 httpx（异步）；或同步代码改用 def 而非 async def，让 FastAPI 放到线程池。
6. 后端：SQLAlchemy N+1	循环里访问 user.orders → 每次都发 SQL。 解：查询时 .options(selectinload(User.orders)) 一次性 join。
7. 后端：MySQL 索引失效	WHERE name LIKE "%xxx%" / WHERE YEAR(time)=2026 → 索引用不上。 解：前缀匹配 LIKE "xxx%" / 改用范围查询 WHERE time >="2026-01-01"。
8. 后端：JWT secret 泄漏	把 SECRET_KEY 硬编码到代码里 push 到 GitHub。 解：立即换 secret + 撤销所有 token。secret 永远只能在 .env 里，加 .gitignore。
9. Docker：镜像越来越大	直接用 python:3.11 → 镜像 1GB+。 解：用 python:3.11-slim / alpine 基础镜像；多阶段构建；.dockerignore 排除 node_modules。
10. Docker：数据库数据丢失	容器删除后 MySQL 数据没了。 解：volumes 必须挂载到宿主机路径。生产环境用云厂商 RDS 不要自建。
11. Git：误 push 大文件 / 密钥	push 后才发现 .env 或 100MB 的视频在 commit 里。 解：git filter-repo（旧版 filter-branch）清除历史；BFG Repo-Cleaner 工具更简单。
12. Git：rebase 冲突	一冲突就慌，乱按 abort。 解：冲突时打开冲突文件，找 <<<<<<<< / >>>>>>>> 标记，手动选择保留版本，git add 后 git rebase --continue。
13. 部署：本地能跑生产报错	90% 是环境变量没配 / Node Python 版本不一致 / 缺系统依赖。 解：用 Docker 统一环境从根本上解决；写 README.md 列环境要求。

14. 部署：HTTPS 证书过期

半夜 12 点用户投诉 — Let's Encrypt 证书 90 天到期没续。
解：用 certbot 自动续期（cron 每月运行）；或上 Cloudflare 全自动 HTTPS。

15. 学习：跟不上技术更新

焦虑感来源。
解：1) 接受这是常态，没人能学完全部 2) 深耕一个方向比啥都懂强 3) 每周固定 1 小时扫"技术周刊"即可。

十、精选资源清单（按方向分类）

每个方向只列 3-5 个最值得的资源。质量大于数量。

前端方向

Vue 3 官方文档	cn.vuejs.org — 中文极好，写得比很多书都好。新人第一站。
TypeScript Handbook	typescriptlang.org/docs/handbook — 短小精悍。TS Playground 配合练。
Vite 文档	cn.vitejs.dev — 主要看 Guide 和配置参考。
Tailwind 官方	tailwindcss.com — Cheat Sheet 收藏起来。
MDN Web Docs	developer.mozilla.org — 浏览器 API 唯一权威来源。
掘金前端专栏	juejin.cn — 中文前端最大社区。每周扫一次。

后端方向

FastAPI 官方教程	fastapi.tiangolo.com — 必读 Tutorial 全部章节，按顺序读完代码量也不大。
SQLAlchemy 文档	docs.sqlalchemy.org — ORM 部分必读。Core 部分用到再看。
Real Python	realpython.com — Python 进阶最佳英文资源（异步/类型提示等深度文章）。
Use the Index, Luke	use-the-index-luke.com — SQL 索引圣经。免费在线书。
Redis 设计与实现	黄健宏著（豆瓣 9.0）— 国人写的 Redis 最好的书。

架构 / 系统设计方向

《Designing Data-Intensive Applications》	DDIA — 系统设计圣经。中文版《数据密集型应用系统设计》。必读，但慢慢读，1 年读 2 遍。
system-design-primer	github.com/donnemartin/system-design-primer — 270k star。系统设计面试经典案例。
ByteByteGo	YouTube 频道 + Newsletter — 系统设计动画化讲解，10 分钟一个主题。
High Scalability	highscalability.com — 收录 Netflix/Uber 等大厂架构演进案例。
《凤凰架构》	周志明著 — 中文写的现代架构最系统的书，开源在线版 icyfenix.cn 。

鸿蒙方向

华为开发者官网	developer.huawei.com/consumer/cn — 唯一权威来源。
HarmonyOS NEXT Codelab	developer.huawei.com/consumer/cn/codelabsPortal — 跟着抄一遍是最快上手方式。
51CTO 鸿蒙专区	edu.51cto.com — 国内系统化视频课。

B 站官方账号	"HarmonyOS 开发者" — 免费课程持续更新。
---------	-----------------------------

工程化 / 工具方向

Learn Git Branching	learngitbranching.js.org — 交互式学 Git 分支。1 小时掌握 90% 的 Git 用法。
Linux Journey	linuxjourney.com — 系统化学 Linux 基础。免费，章节式。
roadmap.sh	前端/后端/DevOps 等技能树。对照查漏补缺。不要试图走完。
Awesome 系列	github.com/sindresorhus/awesome — 每个技术方向都有 awesome-xxx 清单。

中文社区与博客

掘金	juejin.cn — 中文技术社区第一，前端尤其活跃。
V2EX	v2ex.com — 国内程序员的 Hacker News，杂闻+技术。
InfoQ 中文	infoq.cn — 偏架构、企业级。深度文章多。
阮一峰周刊	github.com/ruanyf/weekly — 每周五更新，扫一遍全球技术动态。
左耳朵耗子月报 / 大佬博客	陈皓 / 廖雪峰 / 阮一峰 / Yihui Xie。中文技术博客的天花板。

最后一段话：技术学习最大的浪费不是学错方向，是"反复开始又放弃"。你的基础（PHP + Python + MySQL）已经把 60% 的全栈技能解锁。剩下的 40% 不是新世界，是同一个世界的不同视角。9 个月后回头看今天的纠结，会觉得当时太低估自己了。